Partner App (Hub, Multi-Device)

Purpose and scope

The partner app is the hub staff app. Staff accept restock requests, pick items in expiry order, scan each item and lot, record shortfalls, quarantine bad lots, and seal a cassette ready for a rider. Developer 1 owns the whole partner app.

Devices

- Android tablets (main hub device; kiosk mode possible).
- Android phones (backup for staff on the move).
- Windows and Linux desktops (packing bench or office).
- macOS desktops and iPad or iPhone: only if the hub uses them (see open points).

In scope

- Every hub screen: sign-in, incoming requests, accept and pack, pick list, scan, shortfall, quarantine, cassette ready and labels, offline status.
- One responsive layout for tablet, phone and desktop.
- Device-specific parts: camera scanning on phones and tablets; keyboard-wedge scanner input on desktops; kiosk lock on Android tablets; label printing.
- Offline queue for scans and status changes.
- Tests and release builds for each device type.

Out of scope for this doc

- Design styling (the designer's track); use the design tokens only.
- Backend hub endpoints, stock ledger and restock logic (Developer 1's backend part).
- Customer and rider apps.
- Ops dashboard (separate doc, after the apps).

Words used in the doc.

Term	Plain meaning
Gate	The exact entry point the customer picks for delivery
Gate pin	The gate's map location plus entry instructions
Sweet spot	The area a scooter can start from and still reach a gate within the time budget
Staging node	A legal place where a scooter waits for orders
Lot	One labelled batch of food with its prepared and expiry times
Reservation	A short hold on units so nobody else can buy them
Acceptance	The moment the server confirms the order and starts the 90 s clock

Term	Plain meaning
Leg	One scooter's share of an order
Hub	A small site where scooters restock
Kill switch	A control that turns the 90 s offer off for a zone
p90	The time that 9 out of 10 trips beat

Backend needed before you start

Developer 1 builds these in the backend. The partner app can start on a mock once the contract exists. Real hub data is needed before the scan test.

1. Hub endpoints in the OpenAPI contract

- Listed in the API section below. Paths sit under /v1/hub, matching the rest of the contract (the earlier backend draft used paths without /v1; align them in the first contract review).
- Every response includes server_time.
- Every repeatable write requires an Idempotency-Key header.

2. Mock server (Prism or equivalent)

- Answers every hub endpoint with schema-valid data.
- Scenario header X-Mock-Scenario selects the state.
- Scenarios the app must handle: new restock request arrives; rider arrival time changes; request already accepted by another staff member; item lot wrong; lot expired; temperature failed; partial fill needed; substitute offered; hub closed; duplicate scan; ready sent twice; network down during a scan; live channel drop and reconnect.

3. Live channel mock

- A WebSocket that sends new and changed requests for one hub, with server_time on each event.

4. Fixtures

- One test hub with stock, lots (with expiry and temperature), and a queue of three requests from the rider fixtures.
- Staff test accounts for each role: hub staff, hub lead.

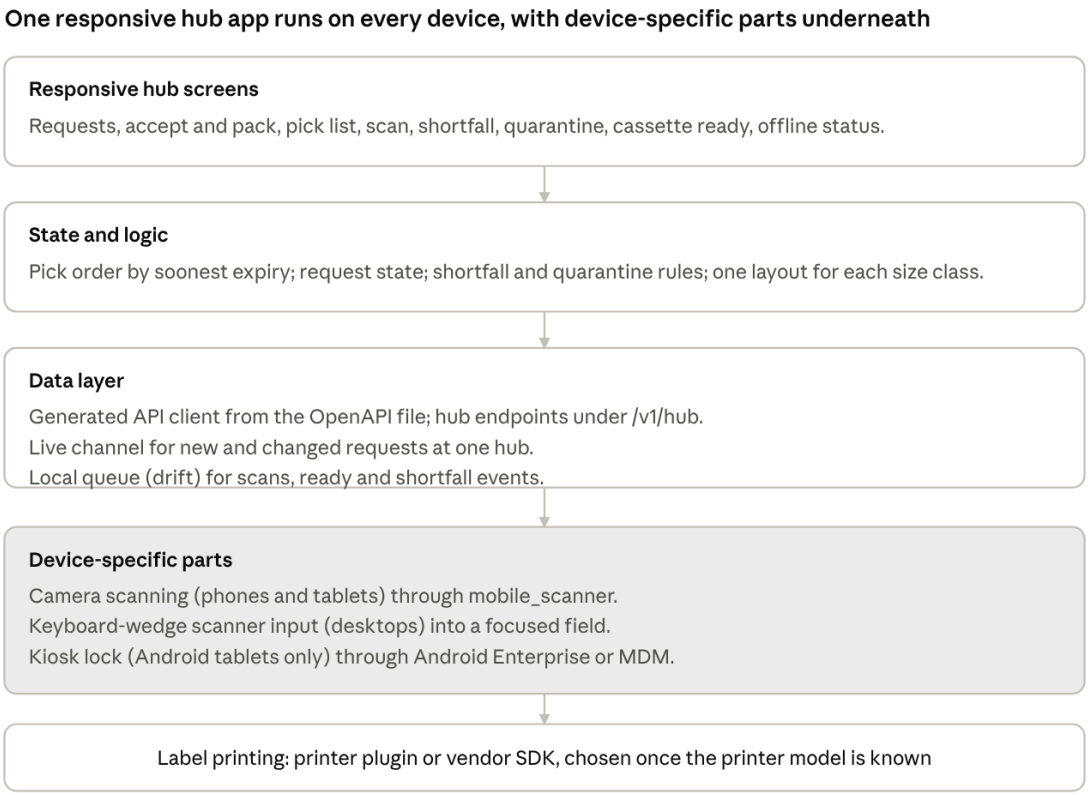
5. Environments

- Mock URL for development, stable for the build.
- Staging URL for device trials, from Phase 3 when real stock data is in place.

What to do while waiting

- Build the request list, pick list and scan screens on the mock.
- Build the desktop keyboard-scanner input and the printer test on real hardware, without waiting for the backend.

Architecture



partner app layers · one responsive app with device-specific parts

One Flutter codebase runs on every device. Each device type adds only its own scanner, printer or lock code. The data layer and the local queue are the same everywhere.

Tech stack and libraries

Flutter for every device. Items marked Verify must be checked on the hub hardware before use.

Need	Choice	Status
Framework	Flutter, same version as the other apps; targets Android, Windows, macOS, Linux, iPad (if used)	Decided
Layout	One responsive layout: LayoutBuilder breakpoints for phone, tablet, desktop	Proposed
State	Riverpod or Bloc (same as the other apps)	Open
Routing	go_router	Proposed
HTTP	dio	Proposed
API client	Generated from OpenAPI	Proposed
Live channel	web_socket_channel	Proposed

Need	Choice	Status
Local queue	drift (SQLite); on desktop the database file sits in the app data folder	Proposed
Camera scanning	mobile_scanner (phones and tablets)	Verify on hub tablets
Desktop scanner input	Keyboard-wedge scanner into a focused text field; Flutter keyboard handling	Proposed; test on hardware
Label printing	printing and pdf packages, or a network printer library (ESC/POS)	Open: needs printer model
Kiosk lock (Android tablets)	Android Enterprise or MDM policy; no app code needed	Ops to choose
Secure token storage	flutter_secure_storage	Verify on Windows and macOS
Desktop window control	window_manager (kiosk-like full screen, close guard)	Verify
Crash reports	sentry_flutter (supports desktop; verify)	Proposed
Distribution	Play internal track (Android); signed installers for Windows and macOS	Proposed

Native parts

- Android tablet kiosk: policy set by MDM, not app code.
- Hardware scanner on an Android tablet, if camera scanning is too slow: a vendor SDK in Kotlin. Only if the test fails.
- Printer: a vendor SDK per platform, only if no plugin works on the chosen printer.

Rules

- Pin the Flutter version in the repo.
- The same screens and state on every device; only the input and print parts differ.
- No platform code outside the device-specific parts.

Tools

- Source and CI: GitHub with GitHub Actions; analyze, unit tests and a build for each target on every pull request.
- Mock and contract: Prism mock server; Schemathesis contract tests against the OpenAPI file.
- API manual testing: Bruno or Postman, with the shared collection.
- Device trials: one Android tablet, one Windows desktop, one macOS desktop if used, and one phone, all at the hub or a test bench.
- Scanner test rig: the hub's real scanner (camera and keyboard-wedge) and the real label printer.
- Distribution: Google Play internal track (Android); signed installers for Windows and macOS.
- Crash reports: Sentry.
- Design handoff: Figma, read-only; tokens exported as files.

- Task tracking: the team's chosen tracker.

Screens and tasks

Each screen lists its data, its states, and when it is done. Every screen works on tablet, phone and desktop.

B1. Project setup and sign-in

- Navigation, theme from tokens, generated API client, environment switch.
- Staff sign-in and role check (hub staff, hub lead).
- Kiosk lock on Android tablets; desktop full-screen mode.
- Done when: staff sign in on the mock and the device-specific setup starts.

B2. Incoming requests

- Data: GET /v1/hub/restock?hub_id=; live channel for new and changed requests.
- Shows each request with the rider's arrival time and its state.
- States: new request; rider arrival time changed; request taken by another staff member; empty queue.
- Done when: a new request appears within two seconds on the mock.

B3. Accept and start packing

- Data: POST /v1/hub/restock/{id}/accept.
- States: accept sent twice (one acceptance only); request withdrawn before accept; hub closed.
- Done when: accepting moves the request to Packing for every device viewing it.

B4. Pick list

- Data: GET /v1/hub/pick-list?request_id=.
- Items sorted by soonest expiry, then by store route order.
- States: item out of stock (shortfall); substitute offered; lot near expiry.
- Done when: the first item to pick is always the one with the soonest valid expiry.

B5. Scan each item and lot

- Data: POST /v1/hub/restock/{id}/scan.
- Camera on phones and tablets; keyboard-wedge input on desktops.
- States: wrong lot (rejected, request stays open); expired lot (rejected); temperature failed (quarantine prompt); duplicate scan (no second count); scanner disconnected.
- Done when: every scanned unit matches the request, and a wrong unit is refused on screen.

B6. Shortfall

- Data: POST /v1/hub/restock/{id}/shortfall.
- Options: partial fill; substitute; send to another hub; note the reason.
- States: partial fill confirmed; substitute refused by the rider (shown on the request).
- Done when: a shortfall is recorded once and the kitchen plan receives it.

B7. Quarantine

- Data: POST /v1/hub/lots/{id}/quarantine (ops-approved path; see open points).
- Shows the reason and asks for confirmation before it is applied.

- Done when: a quarantined lot cannot be scanned again.

B8. Cassette ready and labels

- Data: POST /v1/hub/restock/{id}/ready with the lot numbers in the cassette.
- Prints a label per cassette when a printer is set up.
- States: ready sent twice (one change only); printer offline (show a reprint option); label not printed (request still ready).
- Done when: marking ready records the lot numbers and the rider sees the request as ready.

B9. Offline status and queue

- Shows the count of queued scans and status changes, and the last sync time.
- States: offline; reconnecting; queue sent with some items rejected (shown to staff).
- Done when: a full pick and pack runs offline and syncs in order.

B10. Desktop layout

- Keyboard shortcuts for next item, scan, shortfall and ready.
- Focus stays in the scan field; the scanner input is never lost to a dialog.
- Done when: a full pick and pack works with a keyboard-wedge scanner and no mouse.

B11. Tablet and phone layout

- Large buttons; glove-friendly tap targets; works in store lighting.
- Done when: every action is reachable on a phone in one hand.

Native and device parts

- Camera scanning: mobile_scanner, tested on the hub tablet.
- Keyboard-wedge input: tested on the hub's desktop.
- Kiosk lock: policy set by MDM on Android tablets.
- Printer: plugin or vendor SDK, chosen after the printer model is known.

API endpoints the app calls

All paths are under /v1/hub. Every request sends Authorization: Bearer <token>. Every response includes server_time. Errors use one shape: code, message, retryable. Every repeatable write sends an Idempotency-Key header.

Area	Method and path	Request key fields	Response key fields	Used in
Auth	POST /v1/auth/staff/login	staff_id, credential	access_token, refresh_token, role, hub_id	B1
Requests	GET /v1/hub/restock?hub_id=	hub_id, state (optional)	requests[] (id, rider_arrival_at, items, state)	B2
Live	WS /v1/hub/{hub_id}/stream	token in first frame	request_new, request_changed, server_time	B2

Area	Method and path	Request key fields	Response key fields	Used in
Accept	POST /v1/hub/restock/{id}/accept	none	request (state: packing)	B3
Pick list	GET /v1/hub/pick-list?request_id=	request_id	lines[] (item_id, qty, lot_id, expiry, route_order)	B4
Scan	POST /v1/hub/restock/{id}/scan	lot_code, item_id, scanned_at	unit (accepted or rejected, reason)	B5
Shortfall	POST /v1/hub/restock/{id}/shortfall	item_id, qty_missing, action (partial, substitute, other_hub), reason	request (state), substitute (if any)	B6
Quarantine	POST /v1/hub/lots/{id}/quarantine	reason, evidence	lot (state)	B7
Ready	POST /v1/hub/restock/{id}/ready	lot_numbers[] (cassette contents)	request (state: ready)	B8
Labels	GET /v1/hub/restock/{id}/labels	none	label_data (printable fields)	B8
Sync	POST /v1/hub/events	events[] (event_id, type, payload, device_time)	accepted_ids[], rejected[] (id, reason)	B9
Lots	GET /v1/hub/lots?item_id=	item_id	lots[] (lot_id, expiry, temp_status, qty)	B4, B5

Notes

- The rider sees the request states the hub sets: Packing, Ready, Picked up.
- A scan that is rejected returns the reason; the app shows it and keeps the request open.
- The label data endpoint returns only the fields the printer needs; the printer itself is set up on the device.
- Staff sign-in uses the same OTP or credential method the ops console uses; confirm in the first contract review.
- Field names are a starting point; agree final names with Developer 3 (rider) and the ops console owner, because the same request is shown in several apps.

Hub rules the app must follow

These rules protect stock safety and the handover to riders. They are not optional.

1. **Pick by expiry.** The pick list always shows the lot with the soonest valid expiry first. An expired lot cannot be picked.
2. **Wrong lot is refused.** A scanned lot that does not match the request is rejected on screen; the request stays open.
3. **Temperature failure quarantines.** A failed temperature check opens the quarantine prompt. Staff confirm before it applies; a quarantined lot cannot be scanned again.
4. **Shortfall is recorded once.** Partial fill, substitute or another hub is chosen once and sent once; the kitchen plan gets the shortfall.
5. **Ready records the lots.** Marking ready sends every lot number in the cassette. A repeated tap changes nothing.
6. **Offline scans are queued.** Scans and status changes are saved locally with unique IDs and sent in order. Rejected items are shown to staff, never dropped.
7. **The server decides state.** The app shows the request state from the server, not its own guess.
8. **One request, one acceptance.** Two staff members cannot accept the same request; the second sees it taken.
9. **Desktop focus stays on scan.** A dialog never steals the scanner input; after a dialog closes, focus returns to the scan field.

Testing and release checklist

No build goes to hub staff until every item is ticked on the device type it targets.

- ☐ Unit tests for the pick order, lot rules, shortfall and offline queue pass.
- ☐ Widget tests for every screen state in the screen list pass.
- ☐ Integration tests against the mock server pass in CI.
- ☐ Contract tests against the Developer 1 OpenAPI file pass.
- ☐ Android tablet: camera scanning reads the hub's real lot labels in store lighting.
- ☐ Android tablet: kiosk lock keeps staff in the app.
- ☐ Desktop: keyboard-wedge scanner reads every lot label into the focused field.
- ☐ Desktop: a full pick and pack works with no mouse.
- ☐ Phone: every action reachable in one hand.
- ☐ Label printer prints the cassette label on the real printer, from each device type that prints.
- ☐ Offline: a full pick and pack runs with the network off, then syncs in order with no duplicates.
- ☐ Two staff accept the same request: only one succeeds.
- ☐ Ready tapped twice changes nothing.
- ☐ Temperature failure opens quarantine; the lot cannot be scanned after.
- ☐ Screen-reader labels on controls; text readable at hub lighting; large tap targets.
- ☐ No new crash in the last build in Sentry on each device type.
- ☐ Build shipped to hub internal testers before live hub use.

Open points

These are not decided yet. Developer 1 can make the choice listed, record it here, and tell the other developers if it changes an endpoint.

- Hub device list: how many Android tablets, Windows desktops, Macs, iPads and phones. This sets which device builds are needed.
- Scanner at each hub station: camera, USB or Bluetooth keyboard-wedge, or hardware scanner. Hardware scanners on Android may need a vendor SDK.
- Label printer: model and connection (USB, Bluetooth or network). This sets the printing library.
- Who may quarantine a lot: hub lead only, or any staff member. The endpoint in the screen list assumes hub lead approval.
- Staff sign-in method for the hub (same as ops console or separate).
- Responsive layout breakpoints for tablet, phone and desktop; set during the first device trial.
- Whether iPads are used. If not, iOS is removed from the partner app.
- Whether the partner app runs on Linux as well as Windows and macOS. Default: Windows and macOS only.
- Timing of partner app work: starts after the order and restock endpoints are stable, as agreed.

If a feature or flow changes during the build, record the change in this doc with the date, the reason, and the developer who made it.